

PHASE 4

FitBuddy- AI Fitness Plan Generator using Gemini Models Project Planning

1. Introduction

The Project Planning Phase is the stage in which the requirements and system design of **FitBuddy – AI Fitness Plan Generator** are converted into an organized development roadmap.

In the previous phases, the project idea was identified, requirements were analyzed, and the overall system design was established. The purpose of Phase 4 is to determine **what needs to be developed, in what order it should be developed, what technologies are required, and how the different activities will be organized.**

The FitBuddy source project defines a development workflow consisting of five major milestones:

1. Model Selection and Architecture
2. Core Functionalities Development
3. `routes.py` Development
4. Frontend Development
5. Deployment

Each milestone contains specific activities that contribute to the completion of the application.

2. Purpose of Project Planning

The main purpose of the planning phase is to provide a structured roadmap for developing FitBuddy.

The planning process focuses on:

- Identifying development activities.
- Organizing activities into milestones.
- Selecting required technologies.
- Planning the development environment.
- Planning AI integration.
- Planning backend development.
- Planning database integration.
- Planning frontend development.
- Planning deployment.
- Identifying project dependencies.
- Planning testing and verification.

- Organizing project deliverables.

A structured plan reduces confusion during development and makes it easier to track the progress of the project.

3. Project Planning Overview

FitBuddy is planned as a modular web application combining:

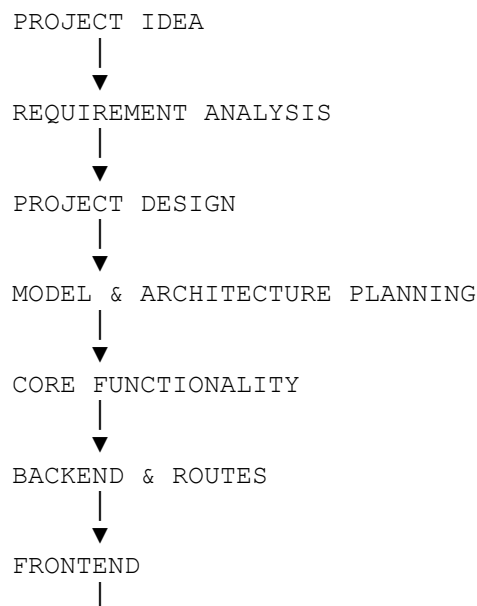
- Python
- FastAPI
- Google Gemini
- SQLite
- SQLAlchemy
- HTML
- CSS
- Jinja2
- Unicorn

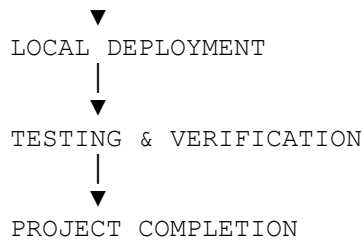
The source project identifies FastAPI for backend routing and logic, Google Gemini for Generative AI functionality, SQLite with SQLAlchemy for persistence, and Jinja2 templates for dynamic frontend rendering.

The development plan therefore divides the project into smaller stages rather than attempting to build the entire application at once.

4. Project Development Strategy

The planned development strategy follows a sequential workflow.





Each stage depends on the successful completion of the relevant previous stage.

5. Project Milestone Plan

The FitBuddy source document defines five major development milestones.

Milestone 1 – Model Selection and Architecture

Activities:

- Activity 1.1 – Research and Select the Appropriate Generative AI Model
- Activity 1.2 – Define the Architecture of the Application
- Activity 1.3 – Set Up the Development Environment

Milestone 2 – Core Functionalities Development

Activities:

- Activity 2.1 – Develop the Core Functionalities
- Activity 2.2 – Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 3 – `routes.py` Development

Activity:

- Activity 3.1 – Writing the Main Application Logic in `routes.py`

Milestone 4 – Frontend Development

Activities:

- Activity 4.1 – Designing and Developing the User Interface
- Activity 4.2 – Creating Dynamic Templates with FastAPI's Jinja2

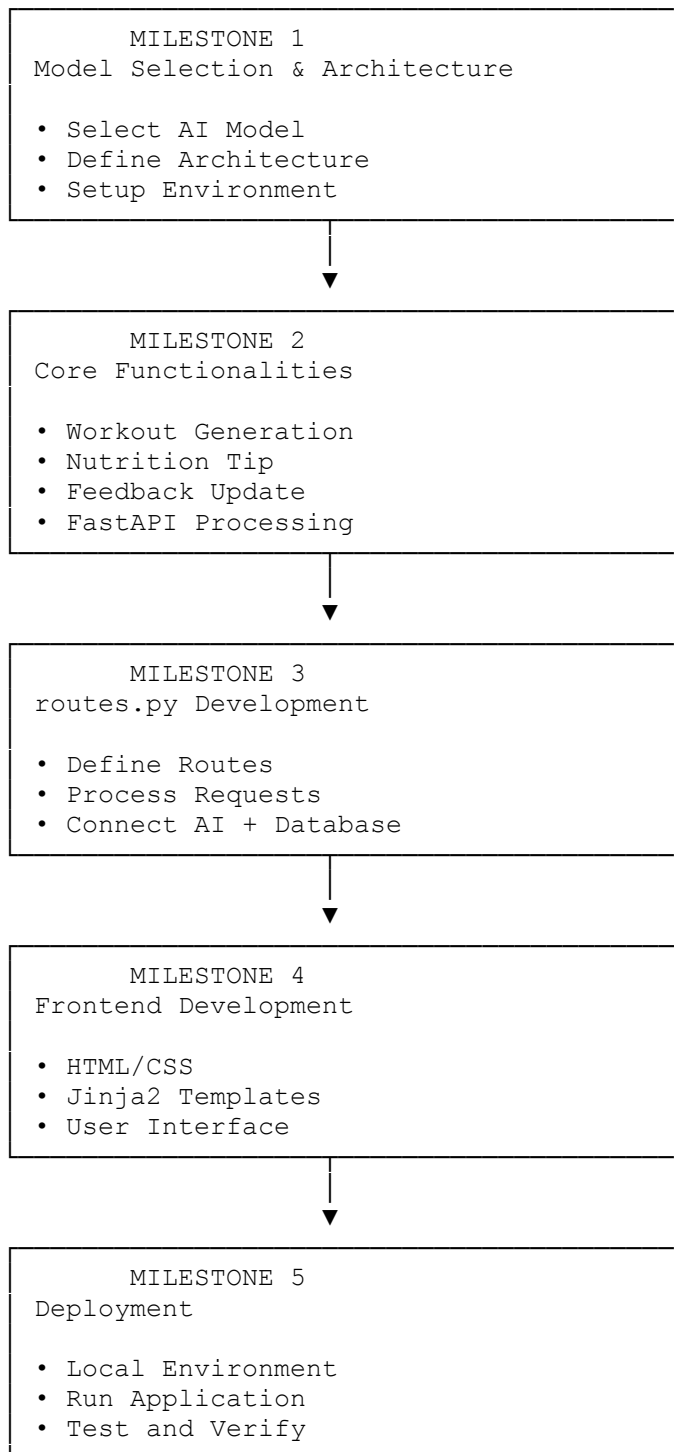
Milestone 5 – Deployment

Activities:

- Activity 5.1 – Preparing the Application for Local Deployment
- Activity 5.2 – Testing and Verifying Local Deployment

This milestone structure is directly defined in the FitBuddy project workflow.

6. Figure 1: FitBuddy Project Planning Roadmap



7. Milestone 1 Planning – Model Selection and Architecture

The first milestone establishes the technical foundation of FitBuddy.

The source project states that the AI model needed to:

- Generate structured 7-day workout plans.
- Provide concise nutrition tips.
- Update plans using user feedback.
- Perform effectively for real-time web interaction.

Therefore, model selection is planned before implementing the main AI features.

8. Activity 1.1 – AI Model Selection

The source project describes experimentation with several Hugging Face models, including:

- T5
- BART
- DistilBERT
- Bloom

The project then documents the selection of Google Gemini models for the production implementation.

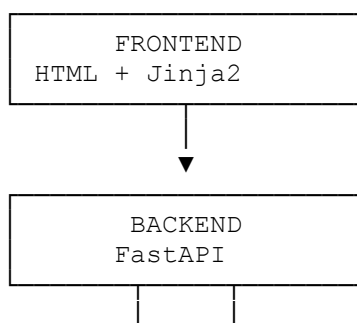
According to the source:

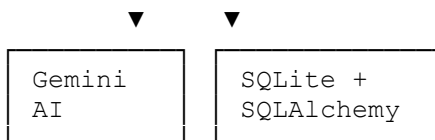
- **Gemini 1.5 Pro** is used for structured workout generation and workout-plan updating.
- **Gemini Flash** is used for fast nutrition/recovery tips.

The planning decision is therefore to separate AI responsibilities according to the required functionality.

9. Activity 1.2 – Architecture Planning

The architecture is planned around four major areas:





The source specifically describes this modular FastAPI architecture with frontend, backend, AI, and database layers.

10. Activity 1.3 – Development Environment Planning

The development environment is planned using Python and a virtual environment.

The source provides the following setup approach:

```
python -m venv fitbuddy-env
```

On Windows:

```
fitbuddy-env\Scripts\activate
```

The documented required libraries include:

```
fastapi
uvicorn
jinja2
sqlalchemy
python-multipart
google-generativeai
```

The source also describes running the development server with Uvicorn and accessing the application through the local browser.

11. Milestone 2 Planning – Core Functionalities

The second milestone focuses on implementing the main capabilities of FitBuddy.

The core functions planned for development are:

1. Workout plan generation.
2. Nutrition/recovery tip generation.
3. Feedback-based plan updating.
4. User and plan storage.
5. FastAPI request processing.
6. AI integration.

These functions form the operational core of the application.

12. Activity 2.1 – Workout Plan Generation

FitBuddy- AI Fitness Plan Generator using Gemini Models

The primary AI functionality is planned around:

```
generate_workout_gemini()
```

The source identifies this function as responsible for generating a personalized 7-day workout plan.

The plan uses information such as:

- Fitness goal.
- Workout intensity.
- User information.

The documented output includes day-wise workout information such as:

- Warm-up.
- Main workout.
- Exercises.
- Sets and repetitions.
- Cool-down or recovery guidance.

13. Nutrition Tip Planning

A separate AI function is planned for nutrition and recovery guidance:

```
generate_nutrition_tip_with_flash()
```

The source describes this function as generating concise and practical nutrition or recovery advice based on the user's selected fitness goal.

Examples described in the source include:

- Hydration advice.
- Protein suggestions.
- General recovery practices.

This keeps the nutrition functionality separate from the main workout-generation process.

14. Feedback-Based Plan Update Planning

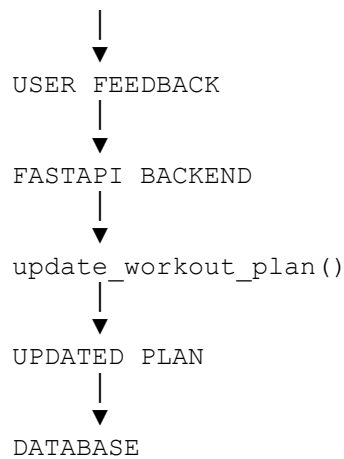
The feedback mechanism is another major functionality planned during Milestone 2.

The planned function is:

```
update_workout_plan()
```

The workflow is:

ORIGINAL PLAN



The source explains that feedback such as requesting more cardio or adding yoga can be used to revise the original plan.

15. Database Planning

The database is planned using:

- SQLite
- SQLAlchemy ORM

The database is intended to store:

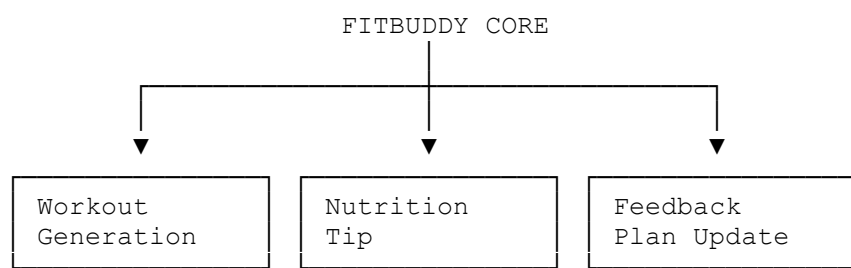
- User information.
- Original workout plans.
- Updated workout plans.
- Feedback-related information.

The source also documents functions such as:

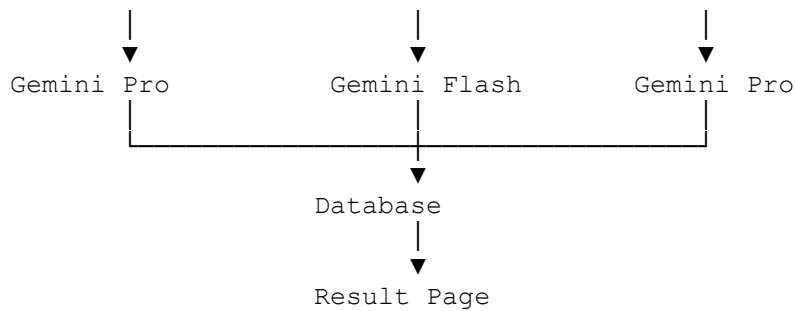
```
save_user()
save_plan()
update_plan()
get_original_plan()
get_user()
```

These functions form the planned database interaction layer.

16. Figure 2: Core Functionality Planning



FitBuddy- AI Fitness Plan Generator using Gemini Models



17. Milestone 3 Planning – `routes.py`

The third milestone focuses on the main application routing logic.

The source describes `routes.py` as the operational layer connecting:

- Frontend templates.
- Gemini AI functions.
- SQLite database.
- User requests.

The route structure is planned around the application's major functions.

18. Planned Core Routes

The source defines the following routes:

Route	Purpose
/	Displays the user form
/generate-workout	Processes user input and generates a plan
/submit-feedback	Updates the workout plan
/view-all-users	Displays users and their plans

These routes provide the main navigation between user actions and backend processing.

19. User Input Processing Plan

The user input form is planned to collect:

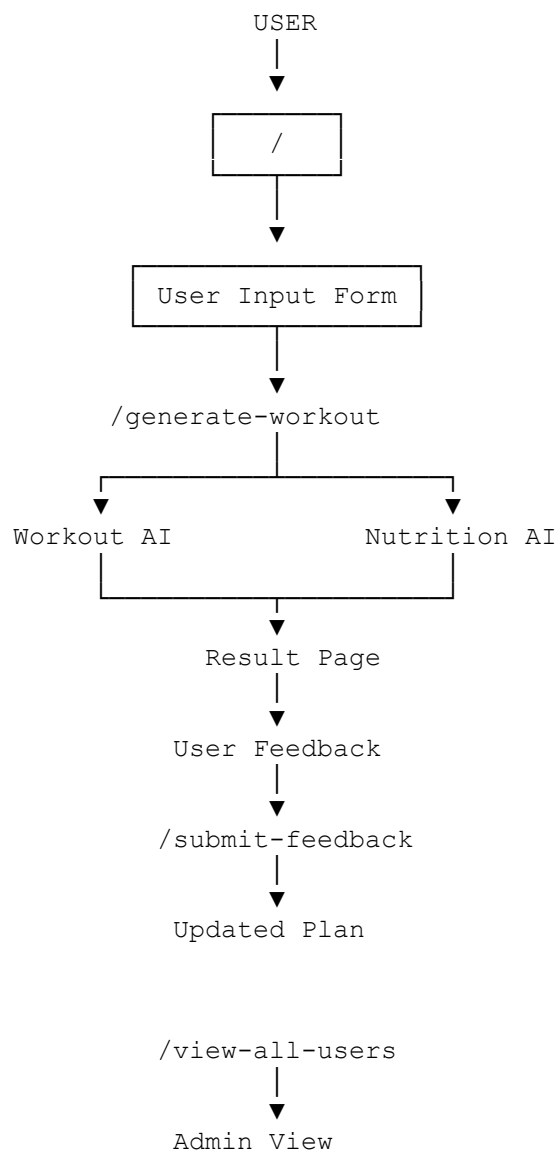
- Username.
- User ID.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.

The source describes FastAPI `Form(...)` parameters for capturing the form values and Pydantic schemas such as:

```
UserInput
FeedbackRequest
```

for structured input validation.

20. Figure 3: Route Planning



21. Milestone 4 Planning – Frontend Development

The fourth milestone focuses on designing and implementing the user interface.

The source describes the frontend as being developed using:

- HTML.
- CSS.
- Jinja2.

The interface is planned to allow users to submit information, receive personalized plans, provide feedback, and support administrative viewing.

22. Activity 4.1 – User Interface Planning

The main input page is planned around the following information:

- Name.
- User ID.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.

The source describes a fitness-oriented design using a gym-themed background, Roboto font, responsive layout, Flexbox, rounded corners, shadows, and button hover effects.

23. Planned Frontend Pages

The documented project identifies three main Jinja2-powered templates:

`index.html`

Used for collecting user input.

`result.html`

Used for displaying:

- User information.
- 7-day workout plan.
- Nutrition tip.
- Feedback form.

`all_users.html`

Used as an administrative view for displaying users and their original and updated plans.

24. Activity 4.2 – Jinja2 Planning

Jinja2 is planned for dynamic rendering of information generated by the backend.

The source describes the use of:

`Jinja2Templates`

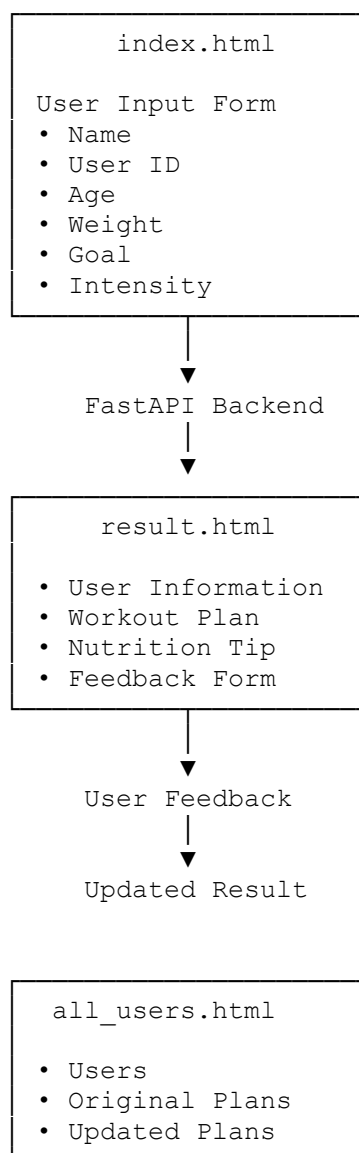
with FastAPI.

The result page uses template variables such as:

```
{{ username }}  
{{ goal }}  
{{ intensity }}  
{{ workout_plan }}  
{{ nutrition_tip }}
```

This allows AI-generated information to be inserted dynamically into the HTML pages.

25. Figure 4: Frontend Planning



26. Milestone 5 Planning – Deployment

The final milestone in the source project focuses on local deployment.

The purpose is to ensure that:

- Dependencies are installed.
- Environment variables are configured.
- The application starts correctly.
- The frontend can communicate with the backend.
- AI functionality can be accessed.
- Database operations work.
- The application can be accessed through a browser.

The source describes this as preparation for possible future cloud deployment.

27. Activity 5.1 – Local Deployment Planning

The deployment plan includes:

Step 1 – Create Virtual Environment

```
python -m venv venv
```

Step 2 – Activate Environment

Windows:

```
venv\Scripts\activate
```

Step 3 – Install Dependencies

```
pip install -r requirements.txt
```

Step 4 – Configure Gemini API Key

The source uses:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

The key can be stored through environment variables or a `.env` file.

28. Activity 5.2 – Deployment Verification Planning

After starting the FastAPI server, the application is planned to be accessed locally.

The documented local address is:

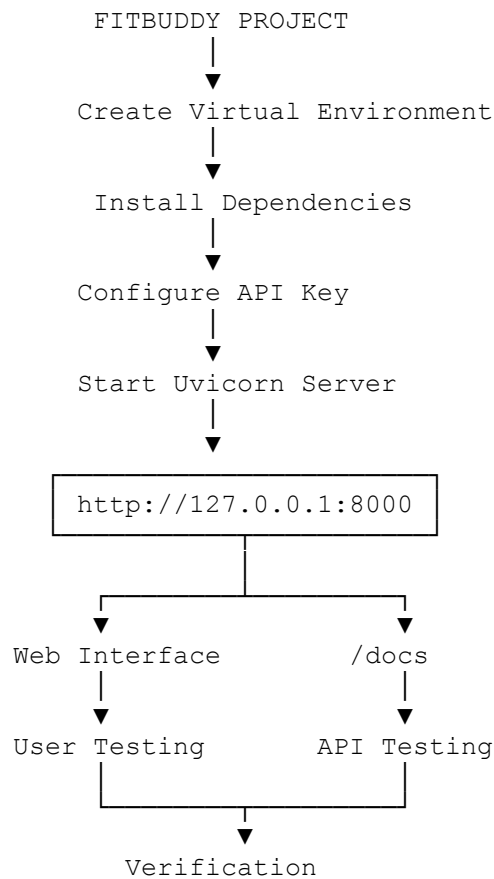
```
http://127.0.0.1:8000
```

The FastAPI interactive documentation is available at:

`http://127.0.0.1:8000/docs`

The `/docs` interface can be used to explore and test the available API endpoints.

29. Figure 5: Deployment Planning



30. Technology Planning

The following technologies are planned for the FitBuddy project:

Technology	Planned Purpose
Python	Application programming
FastAPI	Backend framework and routing
Google Gemini	Generative AI
SQLite	Database storage
SQLAlchemy	Database interaction
HTML	Webpage structure
CSS	User interface styling
Jinja2	Dynamic template rendering
Uvicorn	Local ASGI server

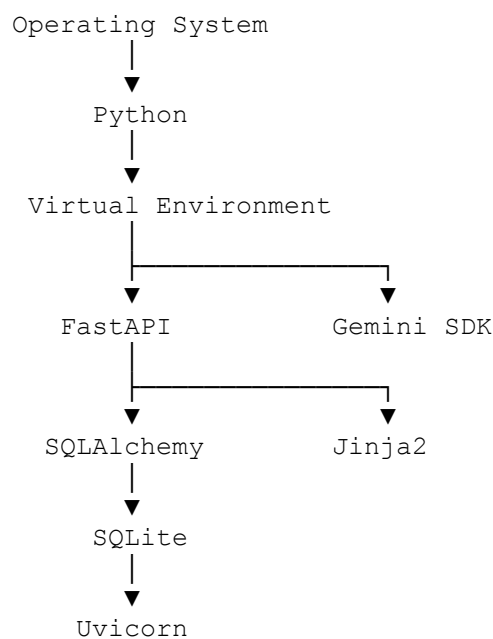
Technology	Planned Purpose
Git	Version control

These technologies correspond to the prerequisites and technical architecture described in the FitBuddy source.

31. Development Environment Plan

The development environment is planned around Python and an isolated virtual environment.

Planned environment:



This structure keeps the project dependencies organized.

32. Resource Planning

The main resources required for the project include:

Software Resources

- Python.
- VS Code or another Python-compatible IDE.
- FastAPI.
- Uvicorn.
- SQLite.
- SQLAlchemy.
- Jinja2.
- Google Gemini API access.
- Git.

Development Resources

- Computer system.
- Internet connection for API access.
- Python package manager.
- Browser for application testing.

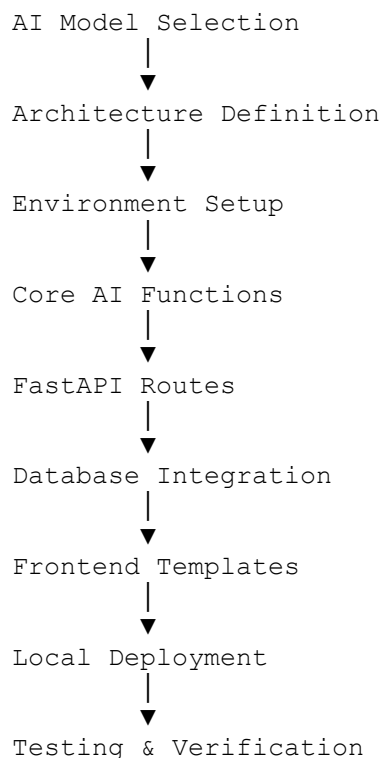
Knowledge Resources

The source project identifies the following prerequisite areas:

- FastAPI framework knowledge.
- Gemini API familiarity.
- HTML, CSS, and Jinja2 skills.
- Python programming.
- Git/version control.
- SQLAlchemy and SQLite.
- Virtual environment setup.
- Uvicorn/ASGI server knowledge.

33. Dependency Planning

The project activities have dependencies.



For example, the frontend result page depends on the backend being capable of generating and returning the workout plan and nutrition tip.

Similarly, the feedback functionality depends on the original workout plan being available for revision.

34. Risk Planning

During planning, possible project risks can be identified.

Risk 1 – AI API Configuration

If the Gemini API key is not configured correctly, AI-generated functionality cannot operate.

Risk 2 – Dependency Problems

Incorrect or missing Python packages may prevent the application from starting.

Risk 3 – Database Errors

Incorrect database operations could affect user and workout-plan persistence.

Risk 4 – Route Errors

Incorrect route definitions may prevent forms from reaching the required backend functionality.

Risk 5 – Template Errors

Incorrect Jinja2 variables or template configuration may prevent generated data from appearing correctly.

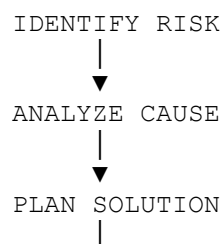
Risk 6 – Local Deployment Issues

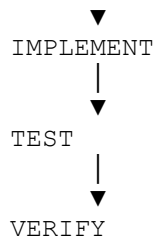
Incorrect Uvicorn configuration or environment setup may prevent the application from running locally.

These risks are considered as part of development planning rather than treated as confirmed failures.

35. Risk Management Approach

The planned approach for managing these risks is:





For example, environment-related issues can be addressed by using a virtual environment and installing dependencies from the requirements file.

36. Testing Planning

Testing is planned throughout development rather than only at the final stage.

The main areas for verification include:

Input Testing

Verify that user information can be submitted.

AI Testing

Verify that workout and nutrition functions return generated responses.

Feedback Testing

Verify that feedback can be processed and used to generate an updated plan.

Database Testing

Verify that users and plans can be stored and retrieved.

Route Testing

Verify the documented routes.

Template Testing

Verify that generated information is displayed through Jinja2 templates.

Deployment Testing

Verify that the application runs at the local address and that `/docs` is accessible.

The source specifically includes local deployment testing and verification in Milestone 5.

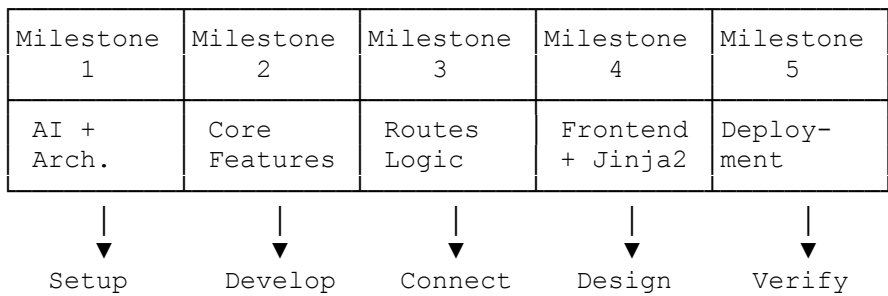
37. Project Schedule Concept

The project can be organized into the following sequence:

Stage	Main Activity
Stage 1	AI model selection
Stage 2	Architecture definition
Stage 3	Development environment
Stage 4	Core AI functionality
Stage 5	FastAPI backend
Stage 6	routes.py
Stage 7	Frontend
Stage 8	Jinja2 integration
Stage 9	Local deployment
Stage 10	Testing and verification

This schedule follows the milestone organization documented in the project source.

38. Figure 6: Project Timeline Concept



39. Project Deliverables Planning

The major planned deliverables include:

Deliverable 1 – AI Integration

Generative AI functionality for workout and nutrition generation.

Deliverable 2 – Backend

FastAPI application with route handling.

Deliverable 3 – Database

SQLite and SQLAlchemy-based data persistence.

Deliverable 4 – Frontend

HTML/CSS and Jinja2-based user interface.

Deliverable 5 – Feedback System

Feedback-based workout-plan revision.

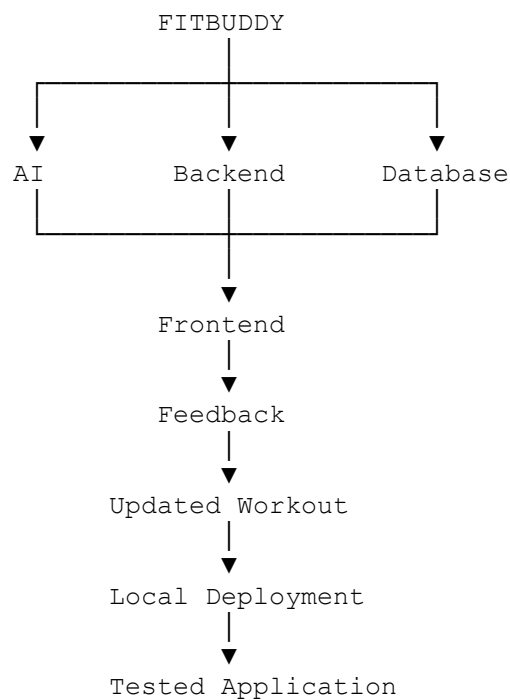
Deliverable 6 – Administrative View

Display of users and original/updated plans.

Deliverable 7 – Local Deployment

A locally running FitBuddy application.

40. Figure 7: Deliverable Relationship



41. Project Progress Tracking

Progress can be tracked by checking each milestone and activity.

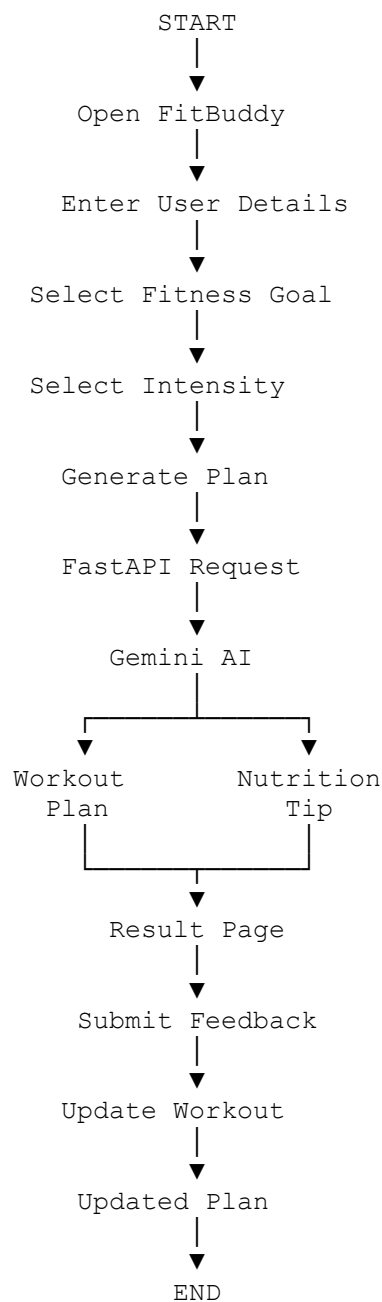
Milestone	Activity	Planned Outcome
1	Model Selection	AI model selected
1	Architecture	System structure defined
1	Environment	Development setup prepared
2	Core Functions	AI and database functions developed
2	Backend	User input and routing implemented

Milestone	Activity	Planned Outcome
3	routes.py	Application routes connected
4	UI	Frontend pages developed
4	Jinja2	Dynamic content connected
5	Deployment	Local application prepared
5	Verification	Application tested locally

This provides a straightforward way to monitor development progress.

42. Planning of the Complete User Workflow

The complete planned user workflow is:



The workflow corresponds to the user interaction and feedback functionality described in the FitBuddy source.

43. Planning for Administrative View

The project also plans for an administrative view.

The /view-all-users route is designed to retrieve user and plan information.

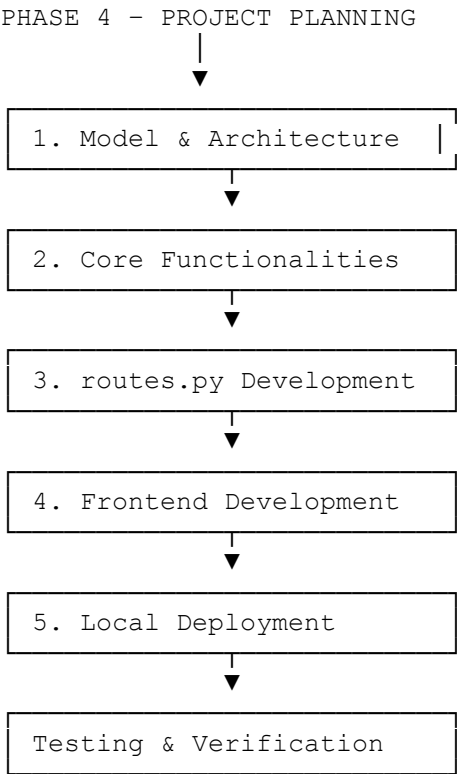
The corresponding all_users.html page is planned to display:

- Username.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.
- Original workout plan.
- Updated workout plan.

The source describes this page as providing an overview of the user database and allowing administrators or developers to monitor how plans have changed following feedback.

44. Project Planning Summary

The complete FitBuddy planning structure can be summarized as:



45. Phase 4 Deliverables

At the end of the Project Planning Phase, the following planning elements are established:

1. Project development roadmap.
2. Five major development milestones.
3. Activity-level development plan.
4. Technology plan.
5. Development environment plan.
6. AI integration plan.
7. Backend development plan.
8. Database plan.
9. Frontend development plan.
10. Deployment plan.
11. Testing and verification plan.
12. Risk considerations.
13. Project deliverables.
14. Progress-tracking structure.
15. User workflow plan.

46. Transition to Phase 5 – Project Development

After completing the planning phase, the project moves into **Phase 5 – Project Development**.

The Project Development Phase will transform the planned architecture, requirements, and activities into an actual working FitBuddy application.

The development work will involve:

- Creating the application structure.
- Implementing the backend.
- Connecting the database.
- Integrating Gemini.
- Developing workout generation.
- Developing nutrition-tip generation.
- Implementing feedback-based plan updates.
- Creating frontend templates.
- Connecting Jinja2 templates to backend data.
- Running the application locally.

The development phase therefore represents the transition from **planning to implementation**.

47. Phase 4 Conclusion

The Project Planning Phase provides a structured roadmap for developing the FitBuddy – AI Fitness Plan Generator.

The plan divides the project into five major milestones: **Model Selection and Architecture, Core Functionalities Development, routes.py Development, Frontend Development, and Deployment**. Each milestone contains specific activities that progressively build the application.

The planning phase also establishes the technologies, development environment, AI integration strategy, backend structure, database requirements, frontend activities, deployment process, testing approach, and project deliverables.

With the project requirements analyzed, system design completed, and development activities planned, FitBuddy is ready to move into **Phase 5 – Project Development**, where the planned components will be implemented as a working application.